

Custom Compiler™ Layout Editor User Guide for Power Devices

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	6
Related Products, Publications, and Trademarks	6
Conventions	6
Customer Support	7
Statement on Inclusivity and Diversity	8

1. Introducing Power Device Tool	9
About Power Device Tool	9
Understanding Basic Connectivity	10
Understanding Power Device PDK	10
Power Device PCell	10
Gate Pad PCell	11
Aluminum Path PCell	12
Abutment	12

2. Getting Started	14
Accessing the Power Device Features	14
PowerDevice Menu Commands	15

3. Placing Power Devices	17
Opening the Power Device Prototype Placer Dialog Box	18
Adjusting Power Device Placer Settings	18
Configuring Parameters	20
Viewing Constant Settings	21

4. Adding and Editing Bonding Points	22
Auto Creating Bonding Points	23
Creating Bonding Points	26
Removing Bonding Points	26

Copying Bonding Points	27
Moving Bonding Points	28
5. Editing Power Devices	29
Replacing Devices	29
Restoring Devices	30
Removing Gate Ring Wires	30
Restoring Gate Ring Wires	32
Creating Inter-Emitter Wires	32
Removing Inter-Emitter Wires	33
6. Simulating Power Dissipation	34
Setting Up Testbenches	34
Performing Simulation	37
Measurement Maps	37
Generating Measurement Maps	38
Showing Measurement Maps	38
7. Power Device Tcl	40
Commands	40
le::pd::createTestbench	41
le::pd::drawLegend	41
le::pd::placer	42
le::pd::recreateMeasureMap	42
ile::pd::autoCreateBondPad	42
ile::pd::copyBondPad	43
ile::pd::createBondPad	43
ile::pd::createInterEmitterWire	43
ile::pd::moveBondPad	44
ile::pd::replaceDevice	44
ile::pd::removeBondPad	44
ile::pd::removeGateRingWire	45
ile::pd::removeInterEmitterWire	45
ile::pd::restoreDevice	45
ile::pd::restoreGateRingWire	46

Contents

ile::pd::showMeasureMaps 46

Preferences 46

About This User Guide

This manual explains how to use the Power Device features in the Custom Compiler™ Layout Editor tool to create arrays and interconnecting wires as PCells, provide electrical models for PCells, and create testbench for electrical and analog simulation in design layouts.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- Synopsys® Custom Compiler™

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .

Convention	Description
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN.</code>
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introducing Power Device Tool

This chapter provides a general overview of the Power Device capabilities in the Custom Compiler Layout Editor, and also briefly describes the basic connectivity, and the Power Design solution customized using the PDK.

This chapter describes the following topics:

- [About Power Device Tool](#)
- [Understanding Basic Connectivity](#)
- [Understanding Power Device PDK](#)

About Power Device Tool

The Power Device tool provides the capability to create and edit power device arrays, alter their sizes, create and edit bonding points, gate pad, and aluminum path (alPath) connecting gates or emitters, and also replaces certain basic power device switch (centralUnit) with snubber devices.

Using the Power Device tool, you can create a testbench for electrical and analog simulation of the switch, conducting simulations, measuring, and displaying current, voltage, and energy at different locations on the layout. The tool lets you optimize your design by looking at the measurements on the layout through changing array sizes and locations, alternate wirings, and creating bonding points.

Custom Compiler offers a creation of power device PDK to simulate electrically equivalent netlist of the layout. The PDK contains PCells for building blocks for layout, their electrical models, various preferences to change resistance and capacitance values in these models and simulation setups. The tool provides capabilities to generate well-formed electrically correct netlist from the layout, simulate it through a test bench and see the results on the canvas. Contact your Synopsys support for getting more information about the power device PDK.

For information on menu commands, see [PowerDevice Menu Commands](#).

For information on Tcl commands and preferences, see [Power Device Tcl](#).

Understanding Basic Connectivity

An IGBT power device layout block essentially contains three terminals, which are Gate (g), Emitter (e), and Collector (c). A basic IGBT switch (centralUnit PCell) also contains these terminals. An arrayed arrangement of these basic switches (powerDevice PCell) is also expected to have these three terminals.

Likewise in a block, when multiple arrays are placed and connected together, the resulting block is expected to have these three terminals. The connections are done to connect gate wires (or terminals) of adjoining arrays through abutments or through explicit aluminum paths. Similarly, emitters are expected to be connected through explicit short creations across adjoining arrays, or through bonding points.

A sample testbench schematic has an element for this entire layout block. The netlisting is expected to generate an electrically correct netlist (as per the arrays, their interconnections with aluminum wires, and so on) ready for simulation.

Understanding Power Device PDK

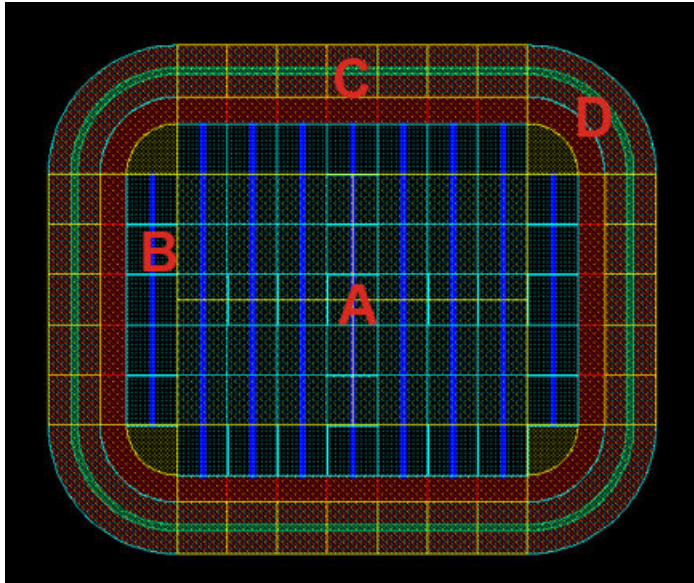
Custom Compiler provides a Power Design solution with specific PCells and customizations in the PDK. These include power device array PCell, aluminum path PCell, gate pad PCell, abutment capabilities across these PCells, their constituent PCells (for example, the array PCell being made up of centralUnit, boundaryUnit, edgeUnit, and cornerUnit PCells), template placer, testbench schematic circuit, and electrical models for each PCell (in order to facilitate netlisting), simulation setup, and so on.

This section covers the following topics:

- [Power Device PCell](#)
- [Gate Pad PCell](#)
- [Aluminum Path PCell](#)
- [Abutment](#)

Power Device PCell

The power device array PCell is implemented as a hierarchical PCell composed to centralUnit, boundaryUnit, edgeUnit, and cornerUnit PCell, as shown in the following figure.

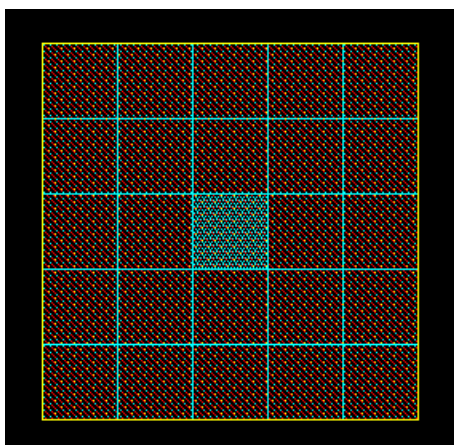


In this power device PCell, (A) 7x5 central units in the center, (B) 5 boundary units on left and right of it, 7 boundary units above and below it, (C) the edge units with (green) gate wire, and (D) 4 corner units on the periphery.

The array size (m x n of centralUnits), sizes of constitutes PCell, and electrical model values are customizable using the Property Editor.

Gate Pad PCell

A gate pad is the main connectivity shape that externally connects to the package pin, and internally connects to the gate ring wire of all powerDevice PCells using aluminum paths. The gate pad is implemented as an array of gatePadUnit PCell. Overall, a gatePad PCell is modeled electrically as a resistive mesh. A 5x5 gatePad PCell is illustrated in the following figure.



Aluminum Path PCell

The aluminum path (alPath) is implemented as an editable PCell guard ring. It can be created using the **Create Guard Ring** command in the Layout Editor. The alPath PCell is composed of constituent alPathSeg PCells; and granularity of segmentation can be controlled by modifying alPath parameters through the Property Editor.

The following figures illustrate an example of alPath PCell and its resistive mesh model.

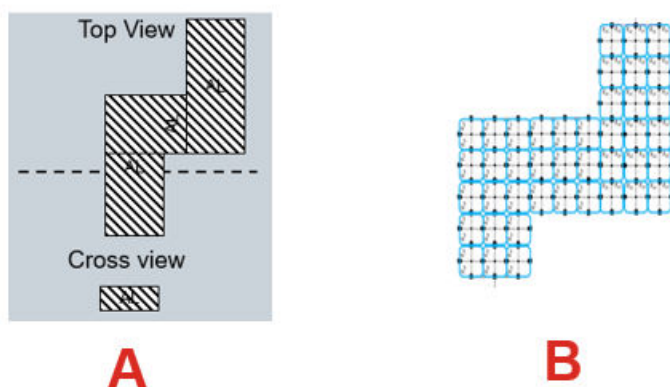
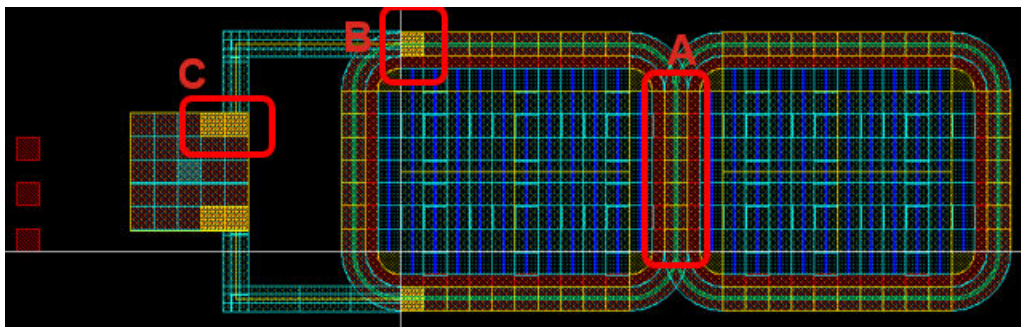


Figure A shows an aluminum path PCell with 3 segments, while figure B shows its corresponding resistive mesh electrical model.

Abutment

For the purpose of propagating connectivity, whenever the gate pad, aluminum path, and power device overlap the corresponding abutment procedure is triggered by the tool. These procedures, which are part of the PDK, provide implementation of reshaping the involved PCells and creating necessary terms, and also assign internal nets as needed.



This figure shows various abutments:

- Across two powerDevice PCells (as depicted in A): When two powerDevice PCells are moved to overlap with each other, the abutment procedure is supposed to move the abutter at a position such that their edge units overlap. The electrical models for abutter and abuttee powerDevice PCells get correctly modified so that netlisting results are electrically correct.
- Across the aluminum path and power device (as depicted in B): When an alPath is moved to overlap with a powerDevice PCell, then its end segment is modified to create correct overlap with edgeUnit inside the power device PCell. The electrical models for abutter alPath and abuttee powerDevice PCells get correctly modified so that netlisting results are electrically correct.
- Across the aluminum path and gate pad (as depicted in C): When an alPath is moved to overlap with a gatePad PCell, then its end segment is modified to create correct overlap with gatePadUnit inside the gatePad PCell. The electrical models for abutter alPath and abuttee gatePad PCells get correctly modified so that netlisting results are electrically correct.

2

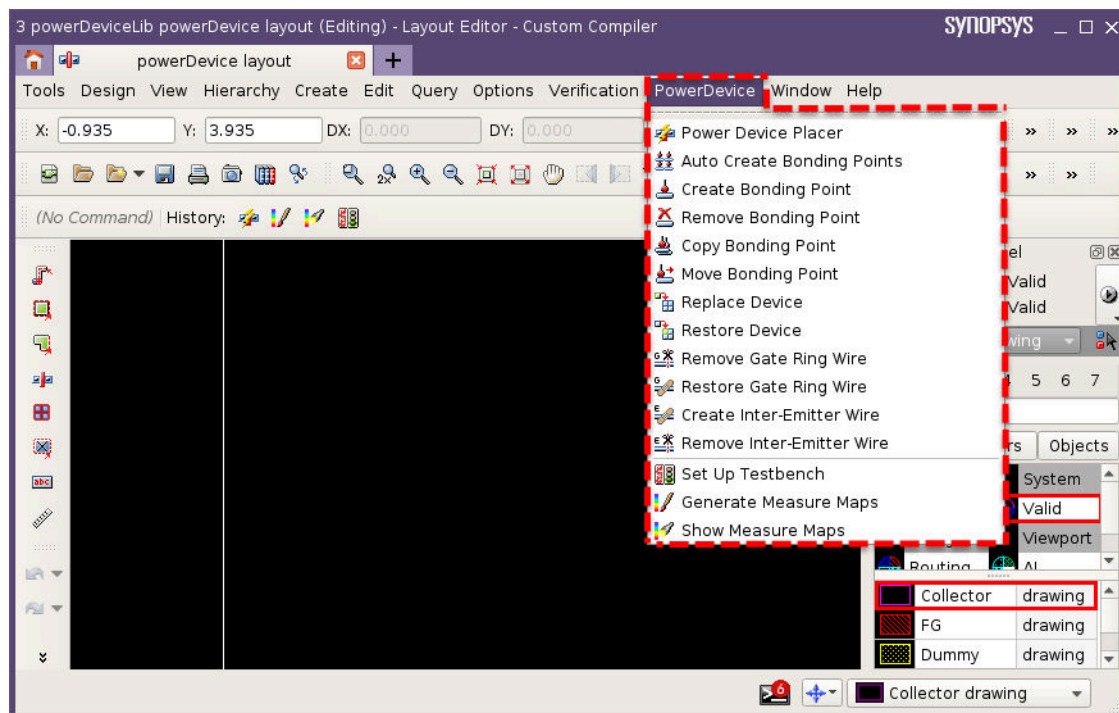
Getting Started

This chapter provides a general overview of the Power Device tool and the menu commands that you would use in the Custom Compiler Layout Editor.

- [Accessing the Power Device Features](#)
- [PowerDevice Menu Commands](#)

Accessing the Power Device Features

The Power Device features are available in the Custom Compiler Layout Editor **PowerDevice** menu.



To display the Layout Editor **PowerDevice** menu:

- After you open a design in the Custom Compiler Layout Editor, choose **Tools > Power Device**. The **PowerDevice** menu appears on the menu bar. It is necessary that you have the Apex license available.

You can create an IGBT layout using the commands under the **PowerDevice** menu, alongside basic layout editing capabilities. There are commands to create testbench, run simulations, and prepare and display measurement maps on the created layout.

Meanwhile, you can also create a Power MOS layout using the commands under the **PowerDevice** menu. See the [Creating and Editing Power MOS Devices](#) application note on SolvNetPlus. For additional assistance, contact your Synopsys Support representative.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

PowerDevice Menu Commands

[Table 1](#) lists the commands in the **PowerDevice** Menu.

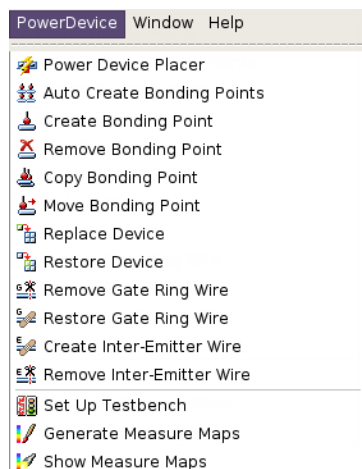


Table 1 *PowerDevice Menu Commands*

Command	Description
Power Device Placer	See Placing Power Devices .
Auto Create Bonding Points	See Auto Creating Bonding Points .

Table 1 PowerDevice Menu Commands (Continued)

Command	Description
Create Bonding Point	See Creating Bonding Points .
Remove Bonding Point	See Removing Bonding Points .
Copy Bonding Point	See Copying Bonding Points .
Move Bonding Point	See Moving Bonding Points .
Replace Device	See Replacing Devices .
Restore Device	See Restoring Devices .
Remove Gate Ring Wire	See Removing Gate Ring Wires .
Restore Gate Ring Wire	See Restoring Gate Ring Wires .
Create Inter-Emitter Wire	See Creating Inter-Emitter Wires .
Remove Inter-Emitter Wire	See Removing Inter-Emitter Wires .
Set Up Testbench	See Setting Up Testbenches .
Generate Measure Maps	See Generating Measurement Maps .
Show Measure Map	See Showing Measurement Maps .

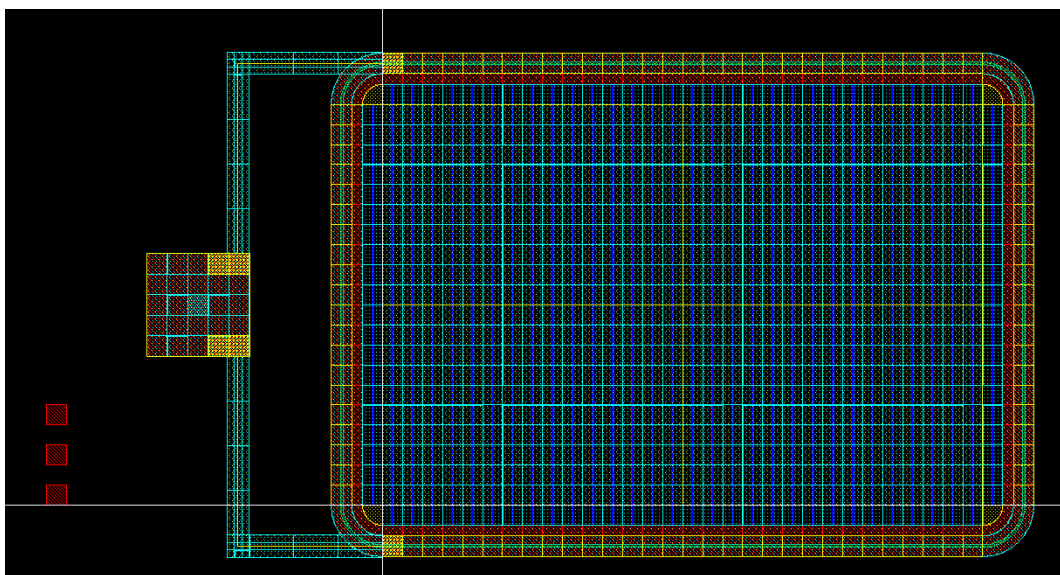
3

Placing Power Devices

This chapter describes the placement of power devices in the layout, and explains how to create and remove power device arrays, as well as alter their sizes.

You can initialize the placement of power devices with a pre-placed template layout using the **Power Device Placer** command.

The following figure shows a sample placemnet with a single 20x30 power device array, along with a 5x5 gate pad placed layout. Three terminals for Gate, Emitter, and Collector on the bottom left. Gate pad (5x5) above it, connecting to power device with aluminum path. Note the abutments across gate pad, aluminum path, and power device.



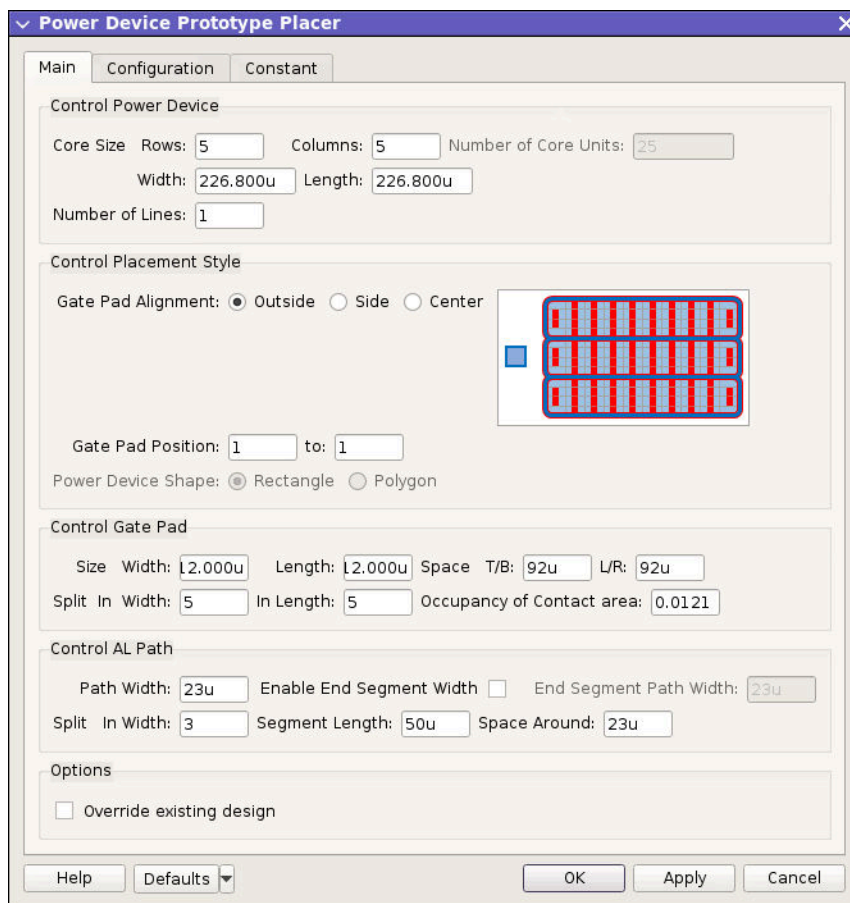
This chapter describes the following topics:

- [Opening the Power Device Prototype Placer Dialog Box](#)
- [Adjusting Power Device Placer Settings](#)
- [Configuring Parameters](#)
- [Viewing Constant Settings](#)

Opening the Power Device Prototype Placer Dialog Box

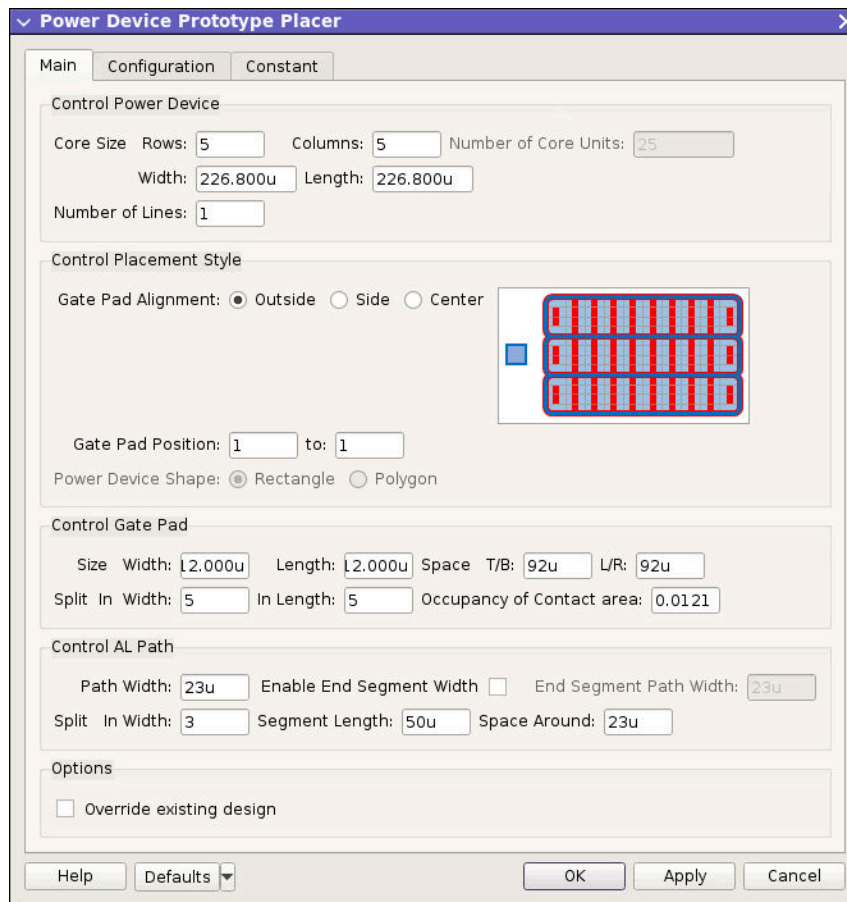
To open the **Power Device Prototype Placer** dialog box:

- With a design open in the Custom Compiler Layout Editor, choose **PowerDevice > Power Device Placer**. The **Power Device Prototype Placer** dialog box appears, showing the prototype placement settings implemented by a power device PDK.



Adjusting Power Device Placer Settings

You can adjust settings in the **Main** tab of the **Power Device Prototype Placer** dialog box.



In the **Control Power Device** section, you can specify the number of power devices, the size of each core power device, and the width and length of core power devices.

In the **Control Placement Style** section, you can adjust the placement of gate pad by choosing the **Outside**, **Side**, or **Center** style. You can also specify the position of gate pad.

Gate pad is essentially a PCell from where the gate terminal connects to the package. It is important to correctly determine the size and location of gate pad, especially in layouts with multiple power device arrays. Thus in the **Control Gate Pad** section, you can specify the size of gate pad, its width, length, and spacings across constituent gate pad units.

Aluminum path, created with the **Create Guard Ring** command, is primarily used to connect the gate pad with gate wires of power devices. It can also be used to connect the emitter plates of the power devices. In the **Control AL Path** section, you can specify the size of aluminum path with description of how it is segmented into basic segments.

Additionally, you can enable the **Override existing design** option to replace the existing placement on the canvas after making changes in the dialog box.

Configuring Parameters

The **Configuration** tab of the **Power Device Prototype Placer** dialog box shows configuration of parameters for placement.

The screenshot displays the 'Configuration' tab of the 'Power Device Prototype Placer' dialog box. The interface is organized into several sections, each with a set of input fields for configuring parameters. The sections and their parameters are as follows:

- CentralUnit Parameters**
 - AL Res: 2.400m
 - TR Res: 40.32
 - AL-WELL Res: 904.772u
 - AL-TR Cap: 0.39424f
- BoundaryUnit Parameters (Top/Bottom)**
 - AL Res (to Edge): 2.400m
 - (to Boundary): 2.400m
 - TR Res: 40.32
 - AL-WELL Res: 2.22206m
 - AL-TR Cap: 0.39424f
- EdgeUnit Parameters (Top/Bottom)**
 - AL(G) Res (to Edge): 2.33739m
 - (to Boundary): 2.46429m
 - FG Res (to Edge): 4.608
 - (to Boundary): 15.1071
 - WELL Res (to Edge): 235.936
 - (to Boundary): 773.507
 - AL(G)-FG Res: 433.036u
 - FG-WELL Cap: 305.76f
 - AL(E)-FG Cap: 29.008f
- CornerUnit Parameters**
 - AL Res: 7.52343m
 - FG Res: 12.8931
 - WELL Res: 660.144
 - AL-FG Res: 134.536u
 - FG-WELL Cap: 855.511f
- AL Path Parameters**
 - AL Sheet Res: 4.8m
 - WELL Sheet Res: 737.3
 - AL-WELL Area Cap: 0.008f
- Gate Pad Parameters**
 - AL Sheet Res: 4.8m
 - FG Sheet Res: 14.4
 - WELL Sheet Res: 737.3
 - CONT Sheet Res: 29.1m
 - FG-WELL Area Cap: 0.39f

Viewing Constant Settings

The **Constant** tab of the **Power Device Prototype Placer** dialog box shows constant settings for placement.

The screenshot shows the 'Constant' tab of the 'Power Device Prototype Placer' dialog box. The dialog has three tabs: 'Main', 'Configuration', and 'Constant'. The 'Constant' tab is active and displays three sections of settings:

- Unit Constants:**
 - Unit Width: 22.4u
 - Unit TR Length: 800n
 - Occupancy of Contact area: 0.0641
 - Boundary Width: 22.4u
 - Boundary TR Length: 800n
 - Occupancy of Contact area: 0.0261
 - Edge FG Width: 35u
 - Edge AL Width: 23u
 - Edge Cont Width: 3u
 - Edge Metal Offset: 0
- Sheet Res/Cap Constants:**
 - AL Res [ohm/um2]: 4.8m
 - FG Res [ohm/um2]: 14.4
 - TR Res [ohm/um2]: 2.88
 - WELL Res [ohm/um2]: 737.3
 - Cont Res [ohm/um2]: 29.1m
 - AL-FG Cap [F/um2]: 0.037f
 - AL-TR Cap [F/um2]: 0.022f
 - FG-WELL Cap [F/um2]: 0.39f
 - AL-WELL Cap [F/um2]: 0.008f
- Limitation Constants:**
 - Min Core Rows: 5
 - Min Core Cols: 5
 - Min Gate Pad Width: 112u
 - Min Gate Pad Length: 112u
 - Min Gate Pad T/B Space: 92u
 - Min Gate Pad L/R Space: 92u
 - Min Gate Pad Split (Width): 5
 - Min Gate Pad Split (Length): 5
 - Min AL Path Width: 23u
 - Min AL Path Space: 23u
 - Min AL Path Split: 3
 - Min AL Path Segment Length: 1u

4

Adding and Editing Bonding Points

This chapter describes the creation and editing of bonding points in the layout.

To connect emitter with package, you may create and edit bonding points on top of basic switches in the power device array PCell.

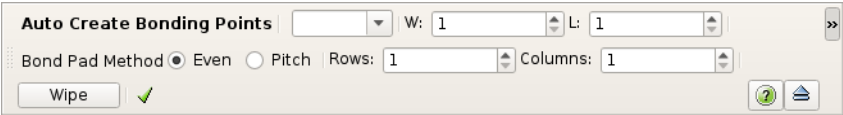
This chapter describes the following topics:

- [Auto Creating Bonding Points](#)
- [Creating Bonding Points](#)
- [Removing Bonding Points](#)
- [Copying Bonding Points](#)
- [Moving Bonding Points](#)

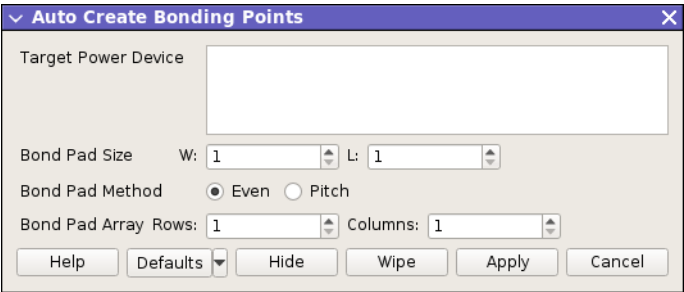
Auto Creating Bonding Points

The **Auto Create Bonding Points** command lets you create a bonding point array to fill in the power device PCell with regularly spaced bonding points and user-specified sizes and spacings. To do this:

1. Choose **PowerDevice > Auto Create Bonding Points** to show the **Auto Create Bonding Points** toolbar.



Click **Eject** to show the **Auto Create Bonding Points** dialog box.



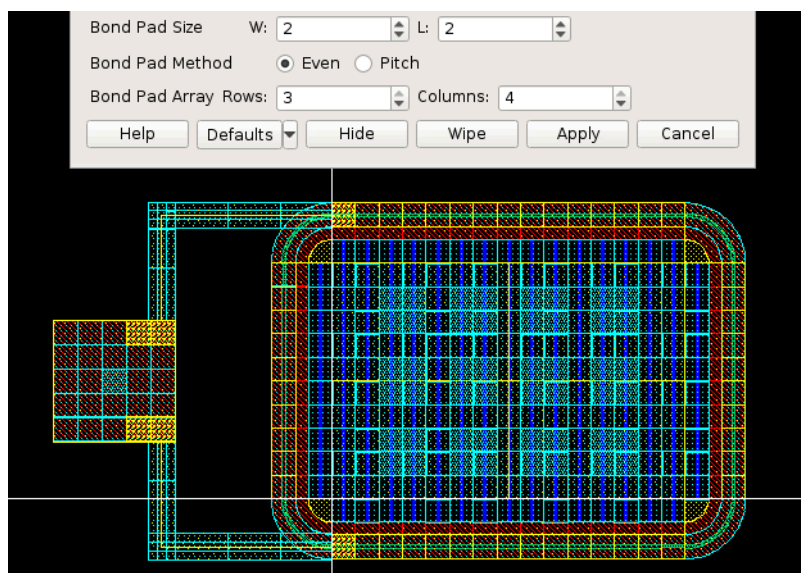
2. In the **Auto Create Bonding Points** dialog box, specify the following:

Option	Description
Target Power Device	Shows the selected power device name.
Bond Pad Size	Specifies width and height for individual bond pads in W and H fields. Tcl preferences: <code>lePdAutoBondPadWx</code> and <code>lePdAutoBondPadWy</code>
Bond Pad Method	Specifies the method for bond pads, either Even or Pitch . Tcl preference: <code>lePdAutoBondPadMethodType</code>
Bond Pad Array	Specifies the rows and columns for bond pad array in Rows and Columns field. Tcl preferences: <code>lePdAutoBondPadRows</code> , <code>lePdAutoBondPadCols</code> , <code>lePdAutoBondPadPitchX</code> , <code>lePdAutoBondPadPitchY</code> , <code>lePdAutoBondPadStartRow</code> , and <code>lePdAutoBondPadStartCol</code>
Defaults	Resets all of the options to your Custom Compiler environment default values.

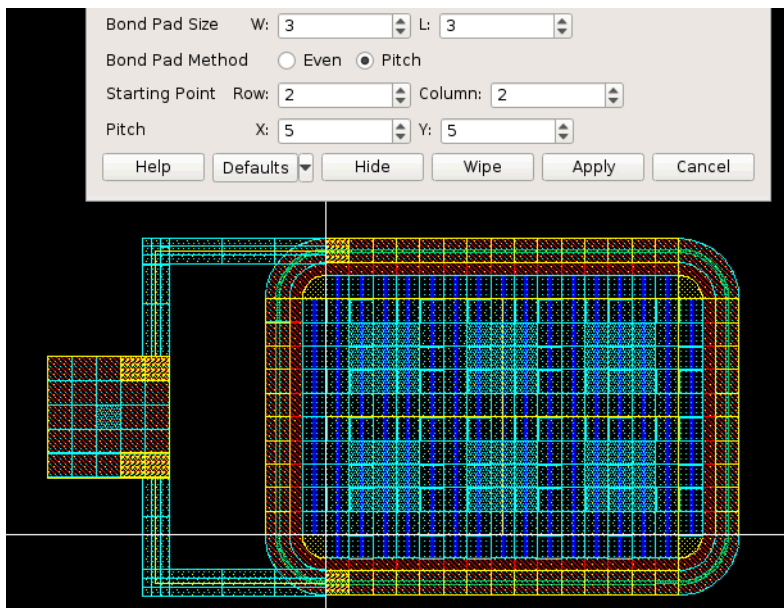
Option	Description
Eject	(Toolbar only) Opens an Auto Create Bonding Points dialog box that lets you configure settings. To switch between the toolbar and the dialog box while the command is active, use the Eject button and the Hide button in the dialog box or press F3.
Hide	(Dialog box only) Closes the dialog box and displays the options in the toolbar.
Wipe	Removes all bonding points on the selected power device.
Apply	Completes the operation.

- Upon entering values for sizes and spacings, the tool shows the to-be-created bonding points on the canvas, which gets committed on clicking the **Apply** button.

The following figure shows a 3x4 bonding point array with each bonding point of size 2x2.



The following figure shows a 3x3 size bonding points with x-pitch of 5 and y-pitch of 5.

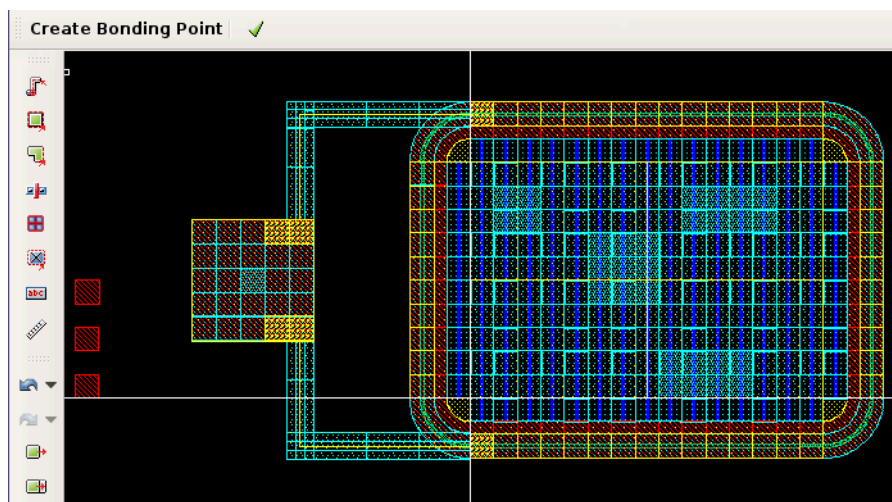


Creating Bonding Points

To create bonding points on top of basic switches in the power device array PCell:

1. Choose **PowerDevice > Create Bonding Point** to show the **Create Bonding Point** toolbar.
2. Draw a region over basic switches and add a set of bonding points by clicking **Apply** (the green check mark) on the toolbar.

The following figure shows bonding points that are created over rectangles, displaying in blue.

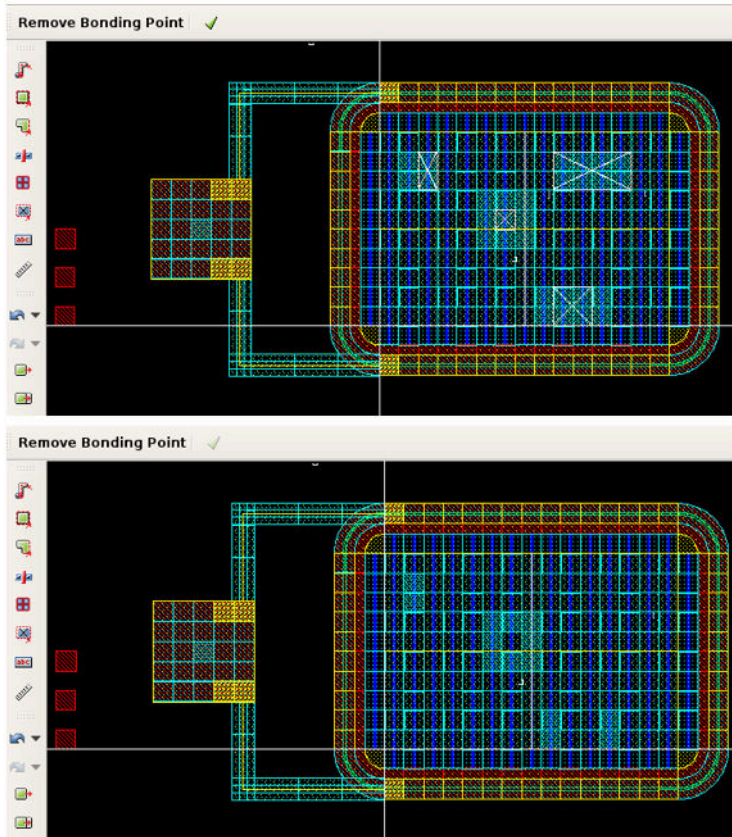


Removing Bonding Points

You can remove bonding points completely or partially from the power device array PCell. To do this:

1. Choose **PowerDevice > Remove Bonding Point** to show the **Remove Bonding Point** toolbar.
2. Area-select a bunch of bonding points and then click **Apply** (the green check mark) on the toolbar to delete them.

The following figure shows partially selected bonding points with crossed annotation, as shown in the upper picture. Upon clicking **Apply**, these portions of bonding points are removed, as shown in the lower picture.



Copying Bonding Points

You can copy bonding points within the power device array PCell. To do this:

1. Choose **PowerDevice > Copy Bonding Point** to show the **Copy Bonding Point** toolbar.
2. Area-select a bunch of bonding points and then click **Apply** (the green check mark) on the toolbar to copy them.

Moving Bonding Points

You can move bonding points within the power device array PCell. To do this:

1. Choose **PowerDevice > Move Bonding Point** to show the **Move Bonding Point** toolbar.
2. Area-select a bunch of bonding points and then click **Apply** (the green check mark) on the toolbar to move them.

5

Editing Power Devices

This chapter describes the editing of power devices in the layout.

This chapter describes the following topics:

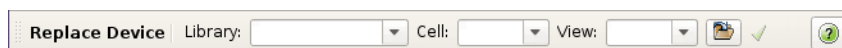
- [Replacing Devices](#)
- [Restoring Devices](#)
- [Removing Gate Ring Wires](#)
- [Restoring Gate Ring Wires](#)
- [Creating Inter-Emitter Wires](#)
- [Removing Inter-Emitter Wires](#)

Replacing Devices

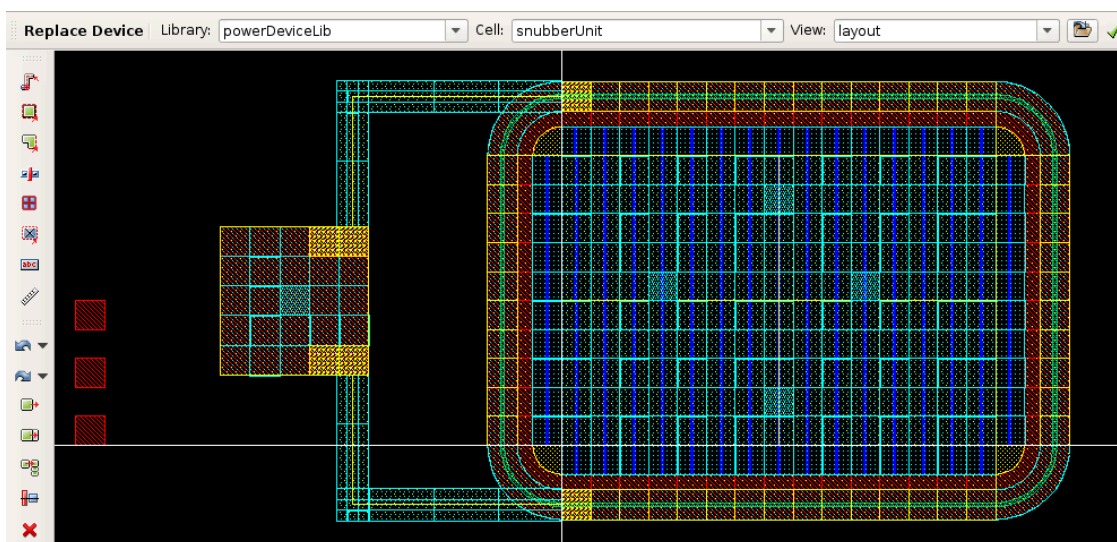
During IGBT device's ON-PFF switching, all its basic power device switches (*centralUnit*) end up switching. The trapped energy in the circuit stray inductance is dissipated in the switching devices, causing a voltage overshoot across the device. This can be avoided by using energy absorbing devices, what is called *snubber* devices.

Snubber devices are plug replacements of a basic switch, and their circuitry helps preventing the internal voltage spikes. For this, the **Replace Device** command lets you substitute the selected basic switch with snubber devices of the specified design database. To do this:

1. Choose **PowerDevice > Replace Device** to show the **Replace Device** toolbar.



2. Click the **Browse** button to select the desired design, which is then appended in **Library**, **Cell**, and **View** (LCV) fields, as shown in the following figure.



3. Upon clicking **Apply** (the green check mark) on the toolbar, the 4 highlighted central units in the figure above are replaced. The *snubberUnit* should have the same bounding box and the number of terminals as that of what it is replacing.

The snubber PCell can implement one of the various energy absorbing techniques , such as decoupling capacitor, voltage clamp, RCD (Resistor Capacitor Diode) charge-discharge, and so on. These PCells are part of the power device PDK.

Restoring Devices

The **Restore Devices** command lets you restore the basic power device switch (centralUnit) to an earlier status for recovery purposes. To do this:

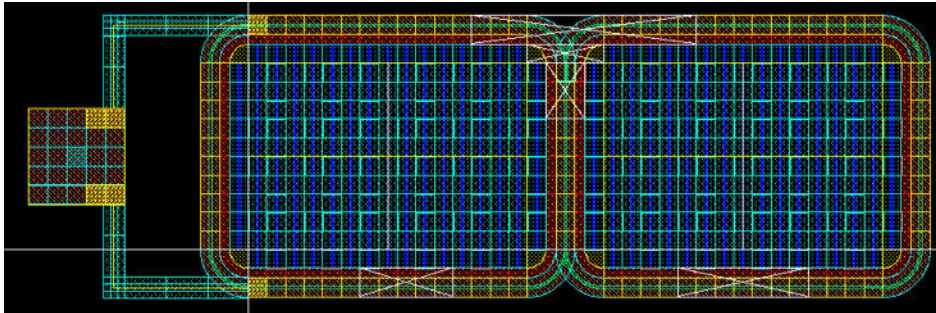
- Choose **PowerDevice > Restore Devices**. The basic power device switch are then restored.

Removing Gate Ring Wires

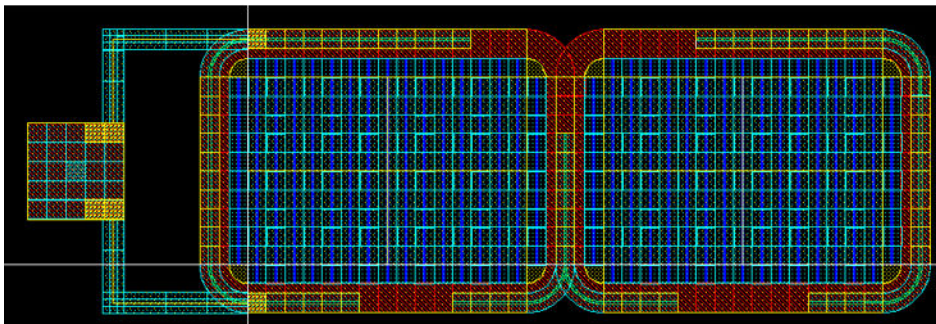
In general, the structure of a power device array has a collector plate in the bottom, an emitter plate on the top, and a gate wire on the periphery. When two power devices are connected, then their collector get shorted automatically. However their emitter needs to be explicitly shorted. This is done by first cutting out the gate wire in the abutted region using the **Remove Gate Wire** command, and then creating an emitter short using the **Create Inter-Emitter Wire** command (see [Creating Inter-Emitter Wires](#)).

To remove a gate wire in the abutted region:

1. Choose **PowerDevice > Remove Gate Wire**. The **Remove Gate Wire** toolbar appears.
2. Area-select to choose the desired gate wires. Selection of edge and corner units where (green) gate wires will be removed is shown with crossed annotation, as illustrated in the following figure.



3. Upon clicking **Apply** (the green check mark) on the toolbar, the gate wires are then removed, as shown in the following figure.



Restoring Gate Ring Wires

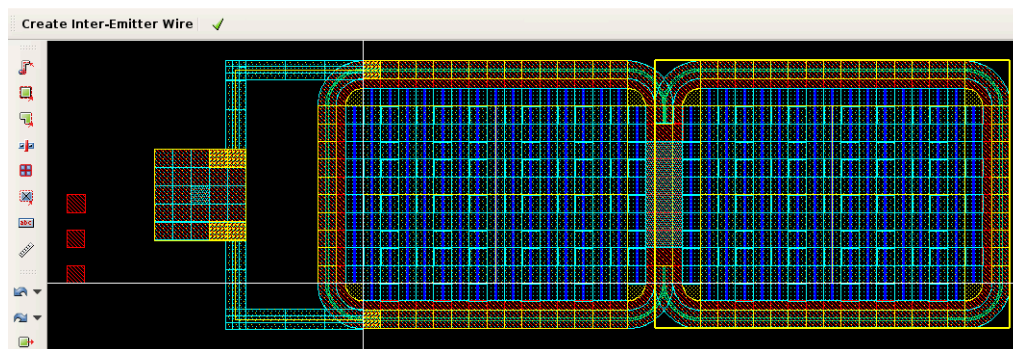
To restore gate wires back into the power device PCells:

1. Choose **PowerDevice > Restore Gate Wire**. The **Restore Gate Wire** toolbar appears.
2. Select the edge or corner units without any gate wire, and then click **Apply** (the green check mark) on the toolbar. The gate wires are then restored.

Creating Inter-Emitter Wires

To create an emitter short:

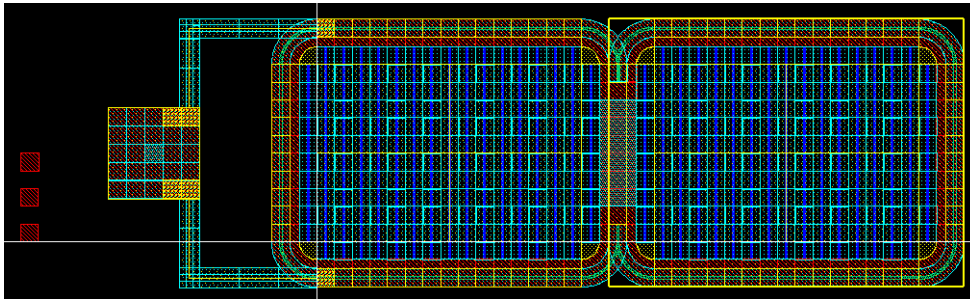
1. Choose **PowerDevice > Create Inter-Emitter Wire**. The **Create Inter-Emitter Wire** toolbar appears.
2. Select the edge units without any gate wire, and then click **Apply** (the green check mark) on the toolbar. The selected edge units are then populated with inter-emitter short.



Removing Inter-Emitter Wires

To remove the inter-emitter short of the selected edge units:

1. Choose **PowerDevice > Remove Inter-Emitter Wire**. The **Remove Inter-Emitter Wire** toolbar appears.
2. Select the edge units where exists the wires you want to remove, and then click **Apply** (the green check mark) on the toolbar.



On the edge units where gate wire is removed, as shown in the figure above, an emitter short can be created as shown in the center of the abutted region using the **Create Inter-Emitter Wire** command (see [Creating Inter-Emitter Wires](#)).

6

Simulating Power Dissipation

This chapter describes the way to set up testbenches, run simulations, and display measurement maps on the created layout.

This chapter describes the following topics:

- [Setting Up Testbenches](#)
- [Performing Simulation](#)
- [Measurement Maps](#)

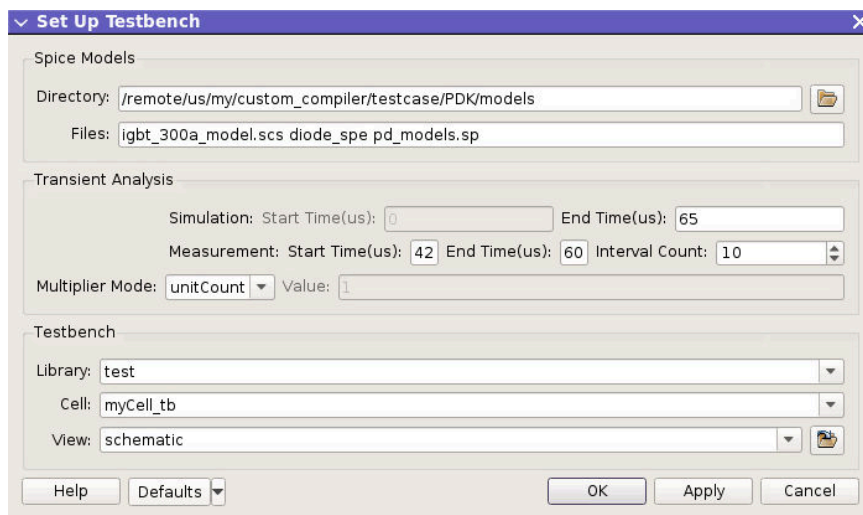
Setting Up Testbenches

The power device layout essentially represents a three terminal switch device. It can be netlisted with electrical models for basic switches, snubber devices, aluminum path, and gate pad. The Power Device PDK provides a built-in schematic that instantiates the power device as a schematic hierarchical block.

The **Set Up Testbench** command generates a `<myCell>_tb` cell sibling to `<myCell>`, which has schematic, customsim state, and config cellviews necessary for managing netlisting and simulation. An example schematic of the testbench is shown in the following

To set up testbenches:

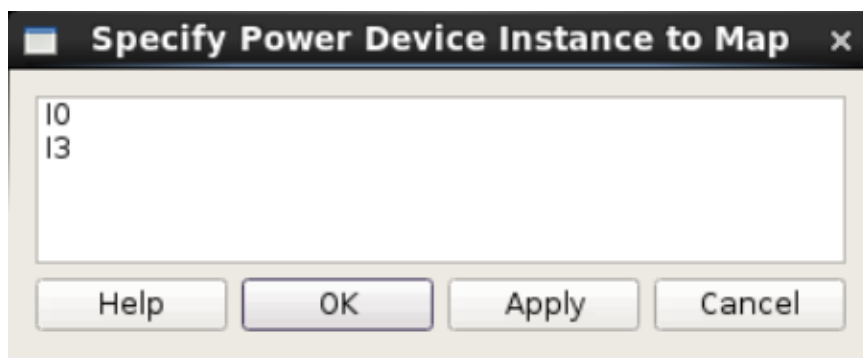
1. Choose **PowerDevice > Set Up Testbench**. The **Set Up Testbench** dialog box appears.



2. In the **Set Up Testbench** dialog box, specify SPICE model files, simulation start and end times, measurement start and end times and the number of steps.

Click the LCV selector button, which is next to the **View** box, to fill the library, cell, and view information for your testbench. The specified testbench (if it does not exist) is created, or if it already exists, then it gets connected to the current power device layout. If you specify your pre-existing testbench, then the tool parses its schematic and identifies candidate instances inside it to match to the power device layout.

If the tool finds multiple matching candidates (ones having three terminals with names G, C, and E), then it lets you select one of these to map to the layout, using the **Specify Power Device Instance to Map** dialog box. The subsequent netlisting then picks the detailed model for this instance by following the layout hierarchy and using the models for the constituent PCells by using its `Model` parameter.



Power device layout is essentially constituted of gatePad, alPath, and powerDevice PCells, which in turn are built with gatePadUnit, alPathUnit, centralUnit, edgeUnit, boundaryUnit, cornerUnit, and concaveCornerUnit.

The electrical models for these are specified with the `Model` parameter, and these models are available as subcircuits in the `pd_models.sp` file. The `pd_model.sp` file is bundled inside the PDK. These models are essentially parasitic models for the gatePad, alPath interconnections, and connections of all central switching unit cells.

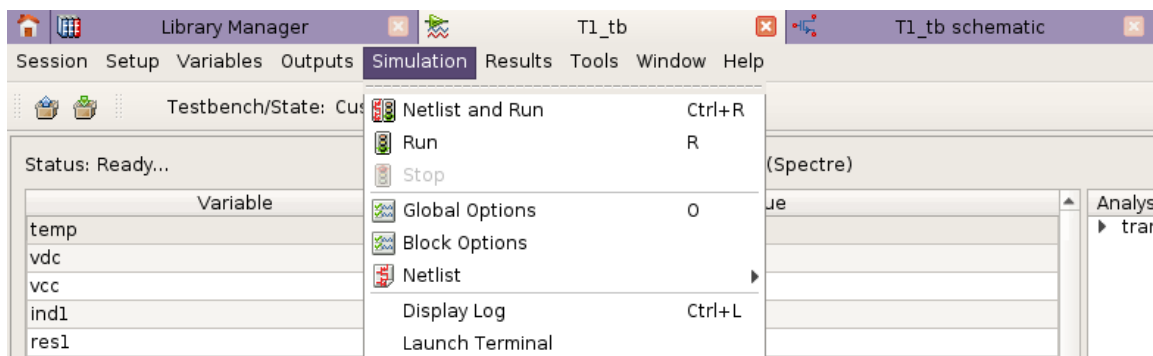
The R and C values for these models are computed from various sheet resistances, lengths, and widths specified in the PDK setup. The central switching unit cell should have a subcircuit model with name `igbt_mod`.

You should specify a model file which defines this model into the **Files** box of the **Set Up Testbench** dialog box. Furthermore, the generated testbench has a diode in it. The model files should also have a subcircuit model for this with name `diode_mod`.

3. Click **OK** or **Apply** to conduct the simulations.

Performing Simulation

CustomSim state cellview gets opened in the PrimeWave Design Environment, from where netlist and simulation runs can be managed, as shown in the following figure.



Measurement Maps

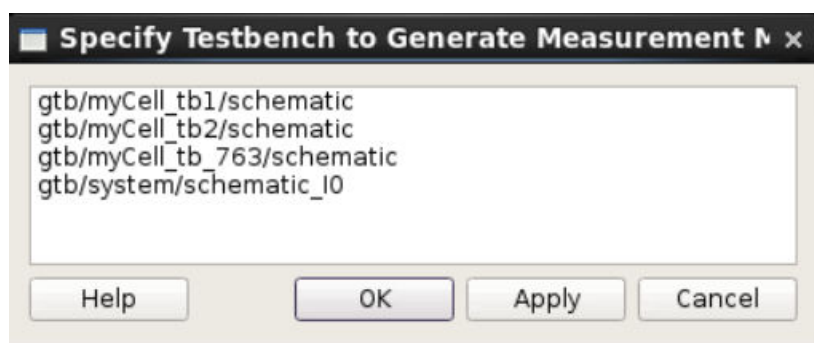
After the simulation is successfully completed, all the required variables are stored in a measurement (`.meas`) file. This contains all the physical data that is of interest to power device layout engineers.

This section covers the following topics:

- [Generating Measurement Maps](#)
- [Showing Measurement Maps](#)

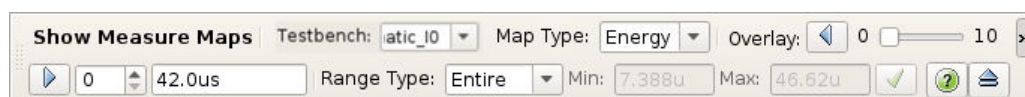
Generating Measurement Maps

The **Generate Measure Maps** command translates the simulation generated *.meas* file into a binary SQL database. This has all the data organized nicely for easy query and display. The *.db* file is created sibling to the *.meas* file. If you have set up multiple testbenches for the current power device layout, you are prompted a **Specify Testbench to Generate Measurement Maps** dialog box, in which you select the testbench schematic for which *.db* file needs to be created.



Showing Measurement Maps

The **Show Measure Maps** command lets you display Energy, Gate Voltage, Collector Current, and Emitter Voltage as heat maps on the power device layout. You can select the map type (physical quantity), timepoint, and display ranges from the **Show Measure Maps** toolbar.



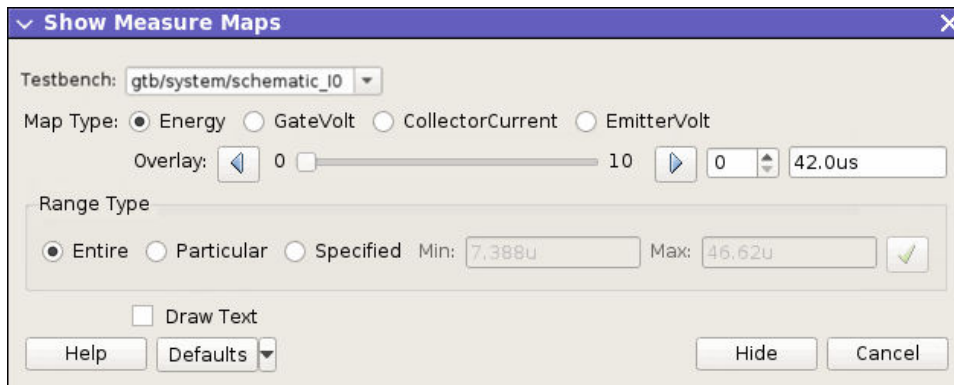
In case of multiple testbenches, you can compare data from different testbenches by switching the testbench. From **Testbench**, you can select the testbench whose data should be displayed in a measurement map.

Map Type provides different physical quantities that can be shown in a measurement map: **Energy**, **GateVolt**, **CollectorCurrent**, and **EmitterVolt**.

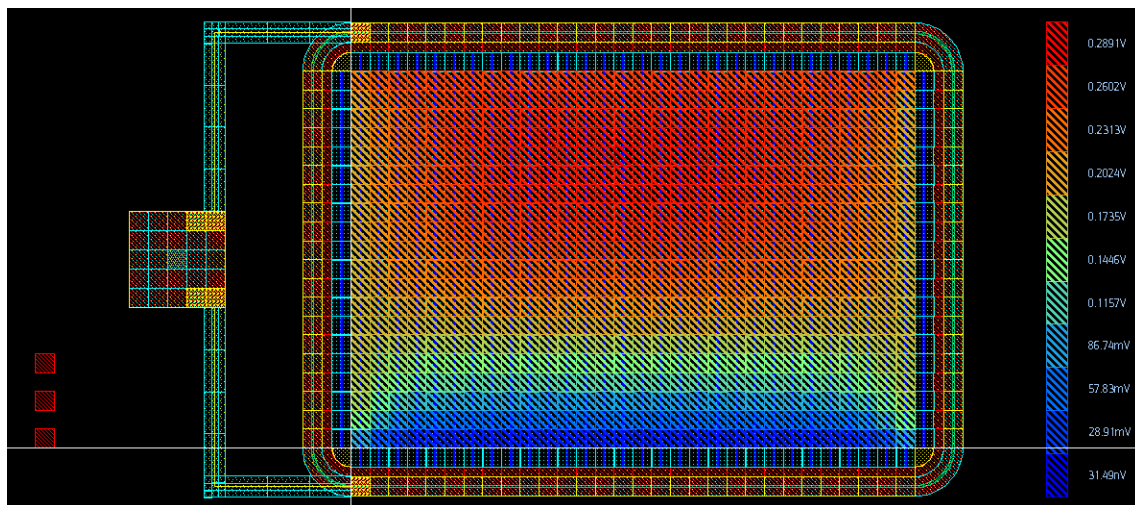
The heat maps can use min-max values of the corresponding physical quantity across the time range of simulation. In **Range Type**, you can set map ranges across all time points

(**Entire**), across current time point (**Particular**), or across user-specified min-max values (**Specified**).

Clicking the **Eject** button on the toolbar shows the **Show Measure Maps** dialog box, as shown in the following figure.



The following figure shows a sample heat map with emitter voltage on the main power device area. The legend of values is drawn on the right.



7

Power Device Tcl

This chapter lists the commands and preferences in the Custom Compiler Tcl `le::pd` namespace. This namespace provides Tcls for power device creation and editing.

This chapter describes the following topics:

- [Commands](#)
- [Preferences](#)

For information on Custom Compiler related Tcl commands and preferences, see Custom Compiler Tcl Reference Manual .

Commands

Interactive commands are listed after non-interactive commands in this section.

This section covers the following commands:

- [le::pd::createTestbench](#)
- [le::pd::drawLegend](#)
- [le::pd::placer](#)
- [le::pd::recreateMeasureMap](#)
- [ile::pd::autoCreateBondPad](#)
- [ile::pd::copyBondPad](#)
- [ile::pd::createBondPad](#)
- [ile::pd::createInterEmitterWire](#)
- [ile::pd::moveBondPad](#)
- [ile::pd::replaceDevice](#)
- [ile::pd::removeBondPad](#)

- [ile::pd::removeGateRingWire](#)
- [ile::pd::removeInterEmitterWire](#)
- [ile::pd::restoreDevice](#)
- [ile::pd::restoreGateRingWire](#)
- [ile::pd::showMeasureMaps](#)

ile::pd::createTestbench

Description

Invokes the **Create Testbench** command to specify simulation start/end time, interval count, and multiplier mode.

Syntax

```
ile::pd::createTestbench [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::drawLegend

Description

Draws a heat map legend on the passed measurement map using the passed minimum and maximum values.

Syntax

```
ile::pd::drawLegend <map: pdMeasurementMap> -minValue <float> -maxValue  
<float> [-suffix <string>] [-precision <int>]
```

Arguments

map

Measurement map object to be passed.

-minValue

Heat map range's minimum value.

-maxValue

Heat map range's maximum value.

`-suffix`

Suffix to be added to heat map values.

`-precision`

Heat map values precision.

le::pd::placer

Description

Invokes the **Power Device Placer** command for power device placement.

Syntax

```
le::pd::placer [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

le::pd::recreateMeasureMap

Description

Invokes the **Generate Measure Maps** command to create a heat map database by parsing the measurement file generated from simulation.

Syntax

```
le::pd::recreateMeasureMap [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

ile::pd::autoCreateBondPad

Description

Invokes the **Auto Create Bonding Points** command to specify values for auto creation of bonding points on the power device central units.

Syntax

```
ile::pd::autoCreateBondPad [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::copyBondPad

Description

Invokes the **Copy Bonding Points** command to copy and paste bonding points within a power device central unit.

Syntax

```
ile::pd::copyBondPad [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::createBondPad

Description

Invokes the **Create Bonding Points** command to create bonding points on the power device central units.

Syntax

```
ile::pd::createBondPad [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::createInterEmitterWire

Description

Invokes the **Create Inter-Emitter Wire** command to create emitter shorts on the edge units of the power device.

Syntax

```
ile::pd::createInterEmitterWire [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::moveBondPad

Description

Invokes the **Move Bonding Points** command to move bonding points.

Syntax

```
ile::pd::moveBondPad [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::replaceDevice

Description

Invokes the **Replace Device** command to replace central units with snubber in the power device.

Syntax

```
ile::pd::replaceDevice [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::removeBondPad

Description

Invokes the **Remove Bonding Points** command to remove bonding points on the power device central units.

Syntax

```
ile::pd::removeBondPad [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::removeGateRingWire

Description

Invokes the **Remove Gate Ring Wire** command to remove gates on the edge units of the power device.

Syntax

```
ile::pd::removeGateRingWire [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::removeInterEmitterWire

Description

Invokes the **Remove Inter-Emitter Wire** command to remove emitter shorts on the edge units of the power device.

Syntax

```
ile::pd::removeInterEmitterWire [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::restoreDevice

Description

Invokes the **Restore Device** command to restore snubber with central units in the power device.

Syntax

```
ile::pd::restoreDevice [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::restoreGateRingWire

Description

Invokes the **Restore Gate Ring Wire** command to restore gates on the edge units of the power device.

Syntax

```
ile::pd::restoreGateRingWire [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ile::pd::showMeasureMaps

Description

Invokes the **Show Measure Maps** command to show a heat map on the power devices, gate pad, and aluminum (al) path, and also show options to change map type, range type, and time points.

Syntax

```
ile::pd::showMeasureMaps [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Preferences

[Table 2](#) lists Custom Compiler lePd general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 2 Power Device Preferences

Name	Type	Scope	Default	Description
lePdAutoBondPadRows	int	cellview	1	Bond Pad rows in the Even mode of auto bonding points creation.
lePdAutoBondPadCols	int	cellview	1	Bond Pad columns in the Even mode of auto bonding points creation.
lePdAutoBondPadMethodType	string	cellview	Even	Bond pad method. Valid values: Even, Pitch
lePdAutoBondPadPitchX	int	cellview	2	Bond Pad columns pitch in the Pitch mode.
lePdAutoBondPadPitchY	int	cellview	2	Bond Pad rows pitch in the Pitch mode.
lePdAutoBondPadWx	int	cellview	1	Width of the individual bond pad.
lePdAutoBondPadWy	int	cellview	1	Height of the individual bond pad.
lePdAutoBondPadStartRow	int	cellview	1	Starting row position of bond pad creation in the Pitch mode.
lePdAutoBondPadStartCol	int	cellview	1	Starting column position of bond pad creation in the Pitch mode.
lePdAutoCreateBondPadDialog Mode	bool	user	false	Command options dialog eject state. Valid values: true, false
lePdMeasureMapsDrawTextPrecision	int	cellview	6	Precision of heat map values shown in Show Measure Maps .
lePdMeasureMapsDrawText	bool	cellview	false	Specifies whether to show or hide heat map values on the power device.
lePdMeasureMapsRangeType	string	cellview	Entire	Heat map range. Valid values: Entire, Particular, Specified
lePdMeasureMapsIntervalNum	int	cellview	0	Number of intervals of time point.
lePdMeasureMapsMapType	string	cellview	Energy	Measurement map type. Valid values: Energy, GateVolt, CollectorCurrent, EmitterCurrent
lePdMeasureMapsRangeMax	real	cellview	10	Range max value for user-specified measurement map.
lePdMeasureMapsRangeMin	real	cellview	0	Range min value for user specified Measurement map.
lePdMeasureMapsSimTime	real	cellview	42	Show Measurement map simulation time value of the time point.

Table 2 Power Device Preferences (Continued)

Name	Type	Scope	Default	Description
lePdShowMeasureMapsDialogMode	bool	user	false	Command options dialog eject state. Valid values: true, false